

PROJECT 33

Design and Implementation of a Database Management System for Training Programme & Certification Management System

By Maneheya (25WU0102149)

Review 1 • Review 2 • Review 3

AGENDA

01

Review 1

Problem, Scope, Objectives, Users, Requirements, ER Diagram, Initial Schema

02

Review 2

3NF Design, Data Dictionary, DDL & Constraints, Sample Data, SQL Queries & Views

03

Review 3

Front-end Application, CRUD Operations, Reports, Testing & Final Deliverables

PROBLEM IDENTIFICATION

Why training centres need a better system

The Situation Today

Training centres run many courses, batches and trainers at once. They must track enrolment, attendance, scores, fees and certificates for every student.

Most of this is still done manually —on paper or in scattered spreadsheets that don't talk to each other.



Wrong certificate eligibility

Students can receive a certificate even without enough attendance or a passing score.



Duplicate enrolments

The same learner can accidentally get enrolled twice in one batch.



Slow, error-prone reports

Attendance, marks and fee data live in separate files, so reports take long and often have mistakes.

PROJECT SCOPE

What this project will and will not cover

In Scope

- ▶ Course, batch, trainer and learner records
- ▶ Enrolment, session and attendance tracking
- ▶ Assessments, scores and fee records
- ▶ Certificate issuing and renewal tracking
- ▶ SQLreports:batchlists,attendance,performance, fees
- ▶ A simple Java/Python app for data entry

Out of Scope

- ▶ Online payment gateway integration
- ▶ Native mobile app version
- ▶ Multi-branch / multi-city support
- ▶ E-learning video / content delivery

PROJECT OBJECTIVES

What we want to achieve

1

Find out what the training centre really needs (requirements).

2

Draw a clear ER diagram of all the data and how it connects.

3

Turn that diagram into real database tables, organised correctly (up to 3NF).

4

Add rules so wrong or duplicate data cannot be saved.

5

Write SQL queries and reports (attendance, fees, results, certificates).

6

Build a simple front-end app so staff can use the system easily.

WHO WILL USE THE SYSTEM?

Four types of users, each with a different role



Administrator

Sets up courses, batches and trainers. Manages the whole system.



Trainer

Takes attendance and enters scores for their sessions.



Student / Learner

Enrols in courses and checks their attendance, fees and certificate.



Office Staff

Handles fee payments and prints certificates.

FUNCTIONAL REQUIREMENTS

1

Admin can add and update courses, batches, and assign trainers to each batch.

2

System stops a batch from taking more learners than its seat limit.

3

Trainers can mark attendance and enter scores for every session.

4

System blocks a learner from enrolling twice in the same batch.

5

Learners can view their own attendance, scores, fees and certificate status.

6

System checks attendance % and score before allowing a certificate to be issued.

7

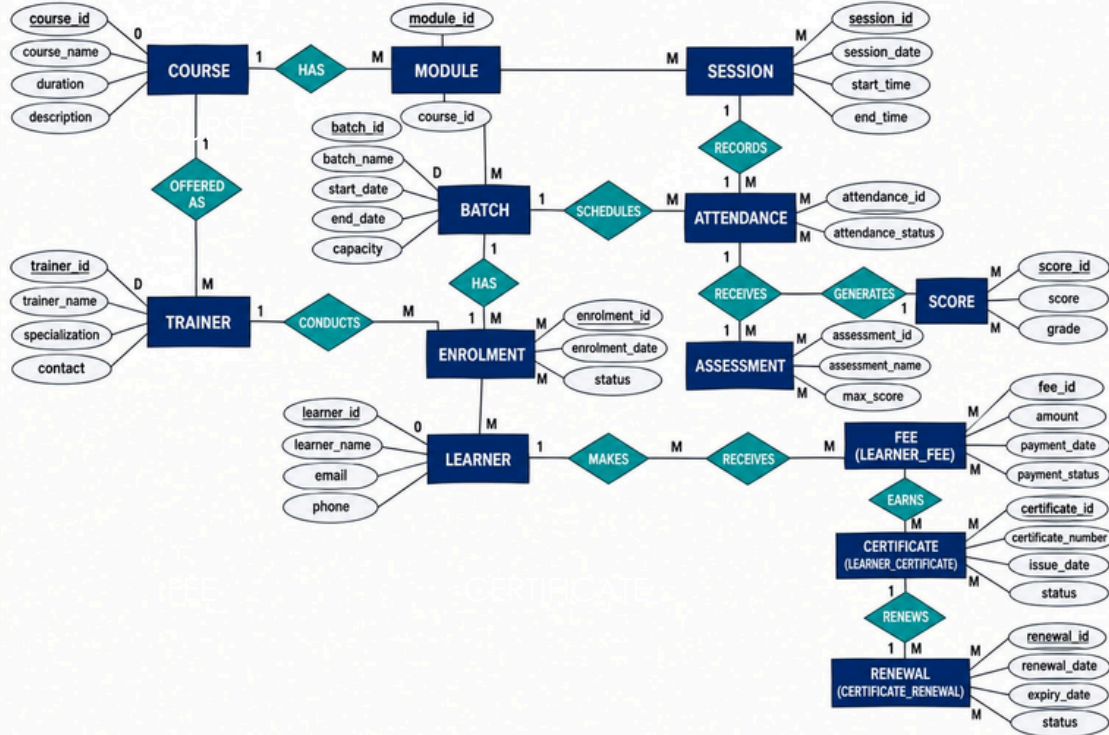
Office staff can record fee payments and issue certificates with a unique number.

8

System can generate reports: batch lists, attendance, performance, fees and renewals.

CONCEPTUAL DESIGN —ER DIAGRAM

13 principal entities covering the full training & certification lifecycle



Key relationships are all one-to-many (1:M). See next slide for connection details.

HOW THE ENTITIES ARE CONNECTED

Every link is one-to-many (1:M)

A Course has many Modules and Batches

A Batch has many Assessments

A Trainer is assigned to many Batches

An Assessment produces many Scores

A Learner can have many Enrolments

An Enrolment produces one Fee record

A Batch has many Enrolments and Sessions

An Enrolment can lead to one Certificate

A Session belongs to one Batch and one Module

A Certificate may have many Renewals

An Enrolment leads to many Attendance records

INITIAL RELATIONAL SCHEMA

Proposed tables, primary keys and relationships

Course	course_id, course_name, duration, description
Module	module_id, module_name, course_id
Trainer	trainer_id, trainer_name, specialization, contact
Batch	batch_id, batch_name, start_date, end_date, capacity, course_id, trainer_id
Learner	learner_id, learner_name, email, phone
Enrolment	enrolment_id, enrolment_date, status, learner_id, batch_id
Session	session_id, session_date, start_time, end_time, batch_id, module_id
Assessment	assessment_id, assessment_name, max_score, batch_id, module_id
Attendance	attendance_id, attendance_status, session_id, enrolment_id
Score	score_id, score, grade, assessment_id, enrolment_id
Fee	fee_id, amount, payment_date, payment_status, enrolment_id
Certificate	certificate_id, certificate_number, issue_date, status, enrolment_id
Renewal	renewal_id, renewal_date, expiry_date, status, certificate_id

REVIEW 1 —SUMMARY

Ready for Review 2

1

Problem, scope, objectives, users and functional requirements are documented.

2

The ER diagram shows 13 entities and how they connect.

3

An initial relational schema proposes tables, primary keys and relationships.

REVIEW 2

Normalized Design, DDL, Sample Data & SQL Queries

Week 11 • 5 Marks

NORMALIZED DATABASE DESIGN (3NF)

Functional dependencies resolved; all tables in Third Normal Form

1NF

Atomic attributes only. No repeating groups. Composite keys split where needed (e.g., Attendance, Score).

2NF

No partial dependency. Non-key attributes fully depend on whole PK. Module depends only on course_id, not batch.

3NF

No transitive dependency. Grade derived from score is kept as stored attribute with CHECK; fee amount not derived from course.

DATA DICTIONARY (KEY TABLES)

Table	Attribute	Type	Constraints
Batch	capacity	INT	NOT NULL, CHECK > 0
Enrolment	learner_id + batch_id	INT, INT	UNIQUE (no duplicate)
Attendance	attendance_status	VARCHAR	CHECK IN ('P','A','L')
Score	score	DECIMAL	CHECK $0 \leq \text{score} \leq \text{max}$
Certificate	certificate_number	VARCHAR	UNIQUE, NOT NULL
Fee	payment_status	VARCHAR	CHECK IN ('Paid','Pending')
Renewal	expiry_date	DATE	CHECK $\geq \text{renewal_date}$

INTEGRITY CONSTRAINTS & BUSINESS RULES

Enforced via PK, FK, UNIQUE, NOT NULL, CHECK and application logic

① Batch capacity limit

Enrolment count cannot exceed Batch.capacity. Enforced by trigger / app check before INSERT.

② No duplicate enrolment

UNIQUE (learner_id, batch_id) on Enrolment table prevents the same learner joining a batch twice.

③ Valid score range

CHECK (score >= 0 AND score <= max_score) on Score; max_score comes from Assessment.

④ Valid attendance status

CHECK (attendance_status IN ('Present','Absent','Late')) keeps values consistent.

⑤ Certificate eligibility

Certificate issued only if attendance $\geq 75\%$ AND average score $\geq$ pass mark (application + stored procedure).

⑥ Unique certificate number

UNIQUE constraint on certificate_number; generated as CERT-YYYY-NNNN sequence.

DDL SCRIPT HIGHLIGHTS

```
CREATE TABLE Enrolment (  
    enrolment_id    INT PRIMARY KEY AUTO_INCREMENT,  
    enrolment_date  DATE NOT NULL DEFAULT (CURRENT_DATE),  
    status          VARCHAR(20) CHECK (status IN ('Active','Completed','Dropped')),  
    learner_id      INT NOT NULL,  
    batch_id        INT NOT NULL,  
    FOREIGN KEY (learner_id) REFERENCES Learner(learner_id),  
    FOREIGN KEY (batch_id)   REFERENCES Batch(batch_id),  
    UNIQUE (learner_id, batch_id)  --no duplicate enrolment  
);
```

SAMPLE DATA & SQL QUERIES

Realistic test data populated; queries demonstrate joins, aggregation, nested queries & views

Batch Lists

JOIN Course, Trainer, Batch with COUNT of enrolments and remaining seats.

Attendance Report

GROUP BY enrolment; calculate attendance % per learner per batch.

Performance

AVG(score), MAX, MIN per assessment; nested query for top performers.

Fee Status

JOIN Fee + Enrolment + Learner; filter by payment_status = 'Pending'.

Certificate Status

View combining attendance %, avg score and eligibility flag.

Trainer Load

COUNT sessions and batches assigned to each trainer.

Renewals Due

Certificates where expiry_date is within 30 days; status = 'Active'.

Views

v_batch_summary, v_learner_progress, v_certificate_eligibility created.

REVIEW 2 —SUMMARY

1

Database normalized to 3NF with documented functional dependencies and data dictionary.

2

DDL scripts include PK, FK, UNIQUE, NOT NULL, CHECK and DEFAULT constraints for all business rules.

3

Sample data loaded; SQL queries and views cover batch lists, attendance, performance, fees, certificates, trainer load and renewals.

4

Database prototype demonstrated and ready for front-end integration.

REVIEW 3

Front-end Application, CRUD, Reports & Final Delivery

Week 16 • 5 Marks

FRONT-END APPLICATION

SimpleJava/Python desktop app connected to the designed database

Course & Batch Setup

Admin creates courses, modules, batches; assigns trainers; sets capacity.

Enrolment

Staff enrolls learners; system blocks duplicates and over-capacity.

Scheduling

Sessions linked to batch + module; date/time entry with validation.

Attendance

Trainer marks Present / Absent / Late per session per learner.

Assessment & Scores

Create assessments; enter scores; auto-calculate grade.

Payment

Record fee payments; update payment_status; generate receipt view.

Certificate Issue

Eligibility check (attendance + score); unique cert number; print.

Reporting

Batch list, attendance %, performance, fees, renewals dashboards.

CRUD, SEARCH, VALIDATION & ERROR HANDLING

Create

Add Course, Batch, Learner, Enrolment, Session, Assessment, Fee, Certificate

Read / Search

Search by name, ID, batch, date range; filter by status; view learner progress

Update

Edit batch details, attendance status, scores, fee status, certificate status

Delete

Soft-delete (status change) preferred; hard delete only for unlinked records

Validation

Input masks, required fields, date ranges, capacity check, eligibility gate

Error Handling

Friendly messages for FK violations, unique conflicts, CHECK failures, connection errors

SYSTEMATIC TESTING

Ensuring data integrity, transaction awareness and indexing

Normalization Check

Verified no partial or transitive dependencies remain in any table.

Constraint Testing

Attempted invalid inserts (duplicate enrolment, over-capacity, bad score) —all rejected.

Transaction Safety

Fee payment + certificate issue wrapped in transactions; rollback on failure.

Indexing

Indexes on frequently searched columns:
learner_name, batch_id, enrolment_date,
certificate_number.

Input Validation

Front-end and DB-level checks for dates, emails, phone formats, numeric ranges.

Report Accuracy

Manual cross-check of attendance %, fee totals and eligibility flags against sample data.

FINAL DELIVERABLES

01

Source Code

Java / Python front-end application
with full CRUD and reporting modules

02

Database Script

Complete DDL (tables, constraints,
indexes, views) + DML sample data

03

Test Data

Realistic multi-batch, multi-learner
dataset covering all edge cases

04

Output Screens

Screenshots of all major screens,
reports and validation messages

05

Project Report

Full documentation: requirements,
design, normalization, testing,
conclusion

06

Viva-Voce

Demonstration of working system and
defence of design decisions

CONCLUSION

1. A complete relational database (3NF) with integrity constraints solves the core problems of manual training-centre record keeping.
2. Business rules —capacity limits, no duplicate enrolment, valid scores, certificate eligibility and unique certificate numbers —are enforced at the database level.
3. SQL queries, views and a simple front-end application enable staff to manage the full lifecycle from course setup to certificate renewal.
4. The system demonstrates normalization, data integrity, transaction awareness, indexing, input validation and systematic testing.

THANK YOU

Questions & Discussion

Maneheyra (25WU0102149)

Project 33 — Training Programme & Certification Management System